Extending Linear Algebra Support to Batched Operations

Document #:	P2901R0
Date:	2023-05-19
Project:	Programming Language C++ SG6, SG19, LEWGI

Reply-to:	Mark Hoemmen <mhoemmen@nvidia.com> Kim Liegeois <knliege@sandia.gov> Christian Trott <crtrott@sandia.gov>

Contents

§ 1 Revision History

§ 1.1 Initial Version 2023-05 Mailing

— Initial version for SG review

§ 2 Abstract

We propose extending P1673 (“A free function linear algebra interface based on the BLAS”) to support “batched” linear algebra, that is, solving multiple independent problems all at once. The initial version of this proposal discusses the interface changes to P1673 that would be needed for batched linear algebra.

§ 3 Motivation

“Batched” linear algebra functions solve many independent linear algebra problems all at once, in a single function call. For example, a “batched GEMM” computes multiple matrix-matrix multiplies at once. Batched linear algebra interfaces have the following advantages.

— By exposing the user’s intent to solve many problems at once, they expose much more potential parallelism and vectorization than a single very small problem has (Dongarra 2018), and amortize the overhead of representing each problem as an argument of a function call. Furthermore, depending on how the interface represents each batch argument, as discussed later, solving many similar problems at once can improve the memory access pattern, reuse common data memory read (see “broadcast” later in this text), and reuse potential common computation.

— They are useful for many different fields, including machine learning, science, and engineering. For a long list of applications that benefit, see Dongarra 2018.

— Hardware vendors such as NVIDIA, AMD, and Intel currently offer optimized software libraries to support batched linear algebra, and hardware features to accelerate it.

— Open-source libraries such as MAGMA and Kokkos offer cross-platform batched linear algebra functionality.

— There is an ongoing interface standardization effort, in which we participate.

It is possible to use existing non-batched libraries, like the BLAS and LAPACK, to solve many small linear algebra problems. However, high-performance implementations of batched linear algebra are not just parallel loops over non-batched function calls. First, non-batched libraries were designed to solve one large problem at a time. For example, they check input arguments for consistency on every call, which could take longer than the actual algorithm for a tiny matrix. Batched libraries can and do amortize these checks. Second, non-batched libraries take each array input and output as a pointer with run-time dimensions and run-time strides. Some problems are small enough that each problem’s

data takes no more space than a pointer to the data, and the problems' dimensions and strides may be known at compile time. Our non-batched linear algebra library proposal P1673 can use mdspan's layout mapping to encode dimensions and/or strides as compile-time constants, but `mdspan` still requires a run-time pointer or data handle. Batching up multiple inputs into a single `mdspan` amortizes the overhead of representing each problem. Third, batched interfaces open up new implementation possibilities, such as interleaving multiple inputs to improve vectorization. Interleaving means that contiguous segments of memory may contain data from multiple independent problems. Even if nested C++17 parallel algorithms worked perfectly when wrapping non-batched functions, this could not easily optimize for this case.

The interface of batched linear algebra operations matters a lot for performance, but may constrain generality. For example, requiring a specific data layout and constraining all matrices to have the same dimensions may make parallelization easier, but applications in sparse multifrontal matrix factorizations may produce dense matrices of different dimensions. Vendors have different interfaces with different levels of generality. For a survey of different interface options, see Relton et al. 2016. This proposal briefly summarizes different interface options and explains why we chose what we did.

The `mdspan` data structure makes it easy to represent a batch of linear algebra objects, and to optimize their data layout. With few exceptions, the extension of P1673 to support batched operations will not require new function names or interface changes. Only the requirements on functions will change. Output arguments can have an additional rank, representing the batch mode. If so, then the leftmost extent will refer to the batch dimension. Input arguments may also have an additional rank to match; if they do not, the function will use ("broadcast") the same input argument for all the output arguments in the batch.

§ 4 Design discussion

§ 4.1 Summary of interface choices

This first version of the proposal does not give complete wording. (See, however, the "Wording sketch" section near the end.) This is because the actual wording would be extremely verbose, and we want to focus at this level of review on how this proposal naturally extends P1673. It does so in the following ways.

P1673's functions (those with BLAS equivalents) can be divided into two categories:

1. "reduction-like," including dot products and norms, that take one or more input `mdspan` arguments (representing vector(s) or a matrix) and return a single scalar value; and

2. "not-reduction-like," which take input and output (or input/output) `mdspan` and return `void`.

For not-reduction-like functions, their batched versions have the same name and take the same number of arguments. We distinguish them by the output (or input/output) `mdspan`, which has one extra rank in the batched case. The input `mdspan` may also have this extra rank. The leftmost extent of the `mdspan` with an extra rank is the "batch extent"; it represents the index of an object (scalar, vector, or matrix) in a batch. All `mdspan` with the extra rank must have the same extent. Those input `mdspan` without the extra rank are "broadcast parameters," that are repeated for all the elements in the batch.

For reduction-like functions, their batched versions have the same name, but return `void` instead of the scalar result type, and take an additional rank-1 `mdspan` output parameter (at the end of the function, which is the convention for output `mdspan` parameters). Each of the one or more input `mdspan` may also have at least one extra rank. As with the non-reduction-like functions, the leftmost extent is the batch extent.

Both cases effectively add a "problem index" to each `mdspan` parameter of the non-batched case. All the problems must have the same dimensions, and the data from different problems are packed into the same `mdspan`. For example, with a matrix-vector multiply $y = Ax$, the different x input are packed into a rank-2 `mdspan` (or rank-1, if this is a "broadcast" input) the different A input are packed into a rank-3 `mdspan` (or rank-2, if this is a "broadcast" input), and the different y output are packed into a rank-2 `mdspan`.

The following section explains the more subtle design choices.

§ 4.2 Discussion of interface choices

§ 4.2.1 Representing dimensions and strides

For a summary of C interface options, see Relton et al. 2016. This technical report establishes vocabulary to describe different kinds of batched BLAS interfaces. The most important interface design choice is how to represent the dimensions and strides of the vector and matrix input(s) and output. We collectively call the dimensions and strides the "metadata." Relton et al. identify three main options.

1. "Fixed": same metadata for all the problems
2. "Variable": "array of problems"; each problem has its own metadata, which may differ
3. "Group": "array of fixed"; multiple instances of fixed, where each instance may have different metadata

Allowing the metadata to vary for different problems makes parallelization and vectorization more challenging. Furthermore, optimizing the fixed case well is a prerequisite for high-performance variable and group implementations. Thus, we focus for now on the fixed case.

Relton et al. 2016 further subdivides the fixed interface into three options, depending on how the interface represents each batch argument.

a. “P2P” (pointer to pointer): each batch argument is an array of arrays (each element of the outer array is a pointer to an element of the batch).

b. “Strided”: each batch argument is packed into a single array, with a fixed element stride (space) between the start of each input in the batch.

c. “Interleaved”: for example for a batched matrix *A*, given that the leftmost extent is the batched extent, then the elements `A[0,i,j]`, `A[2,i,j]`, …, `A[K-1,i,j]`, i.e., the `i, j` elements of all the matrices in a batch are stored contiguously. (This can be generalized, for example, to some fixed SIMD width number of problems (such as 8) having their `i, j` elements stored contiguously.)

Different vendors offer different options. For example, NVIDIA’s cuBLAS includes both P2P (`*Batched`) and strided (`*StridedBatched`) operations, and its CUTLASS library supports many variations of strided and interleaved.

The P2P interface would require extra packing and unpacking of pointers, and therefore extra overhead. In practice, users often want to represent a batch as a “pre-packed” array with a particular layout. If they only had a P2P interface, they would waste time and code setting up the array of pointers. Even though P2P could be used for the fixed interface, it is more useful for the variable or group interface. Thus, we exclude the P2P option for now.

The `mdspan` class lets us naturally combine “strided” and “interleaved” into a single interface by using different, possibly custom mdspan layouts. Each batch parameter would become a single `mdspan`, with an extra rank representing the batch mode (the index of a problem within the batch).

§ 4.2.2 Representing scaling factors (alpha and beta)

Some BLAS functions take scaling factors. For example, a single matrix-matrix multiply computes `C = beta * C + alpha * A * B` for matrices `A`, `B`, and `C` and scalars `alpha` and `beta`. Batched linear algebra has a design choice: should the different problems in a batch use the same or different scaling factors? Different vendor libraries make different choices. For example, the `*StridedBatched*` functions in NVIDIA’s cuBLAS take an array of scaling factors, one element for each problem. Intel’s oneMKL’s “group” interface uses the same scaling factor(s) for all the problems in a single fixed group, but let the scaling factor(s) vary for different groups.

P1673 expresses scaling factors for the non-batched case with an accessor `accessor_scaled`, that users access mainly by calling the `scaled` function. For example, `scaled(alpha, A)` represents the product of the (scalar) scaling factor `alpha` and the matrix `A`, as an `mdspan` that defers this multiplication until the actual kernel.

If we want to use the same scaling factor for all the problems in a batch, we can use `accessor_scaled` and `scaled` without interface changes. However, if we want to use a different scaling factor for each problem, our only choice is to change all the function interface to take additional separate `mdspan` parameters for the scaling factors.

One may wonder why we couldn't just change the `scaled` function to take an `mdspan` of scaling factors. The issue is that the `scaled` function needs to return a single `mdspan` that represents the deferred multiplication. Only an `mdspan`'s accessor can affect the elements of the `mdspan` by deferring a multiplication in this way. However, by the time `mdspan::operator[]` reaches the accessor, the index information from the layout mapping about which scaling factor to apply would no longer be available. If we made `scaled` return something other than an `mdspan` and generalized the function to take arguments more generic than `mdspan`, then that would violate the design principle expressed in P1673, that the only generality allowed for vector or matrix arguments is the generality that `mdspan` itself offers. P1673 is not a general linear algebra library (e.g., it's not for sparse linear algebra); it's a C++ BLAS interface.

Note that for applying a single scaling factor to all the elements of a batch, the existing `scaled` function works just fine. (We would only need to allow rank-3 `mdspan` arguments.)

4.2.3 Conjugate, transpose, and triangle arguments

Dongarra 2018 proposes that the different problems in a batch could take different conjugate, transpose, triangle, or diagonal-interpretation (explicit or implicit unit) arguments. However, not all vendor libraries support this. Furthermore, changing these arguments actually changes the algorithm in a way that this not always amenable to optimizations like vectorization. For these reason, we require that all the problems in a batch have the same conjugate, transpose, triangle, and diagonal-interpretation arguments.

4.2.4 Representing the result of a reduction (dot product or norm)

The Batched BLAS interface specification (Dongarra 2018) omits "reduction-like" operations – dot products and norms – that return a single value.

The original P1673 design had reductions write to an output reference (or rank-0 mdspan), with the intent that this could be generalized to an output rank-1 (the batch mode) mdspan. LEWG and previous Study Groups asked P1673 authors to make reductions look like `std::reduce`: returning a value, instead of writing to an output argument. This interface is easier to understand and more consistent with the Standard Library for the non-batched case. However, it means that the batched interface cannot be made fully consistent with the non-batched interface. (This is because we do not permit P1673 or its extensions to allocate and return arrays of elements. It is an essential feature of P1673, as it was of the BLAS and LAPACK, that it can be implemented and used without dynamic memory allocation. Implementations may still choose to do dynamic memory allocation internally, but they are not required to do so.)

This gives us two design options.

1. Overload reduction functions for the batched case to return `void` and take an output `mdspan`.

2. Omit reductions from the batched interface.

We favor the first approach: adding overloads of P1673 reduction-like functions that return `void` and write the reduction results to an output `mdspan`. This has the disadvantage that the batched and non-batched versions of the same function would no longer take the same number of arguments. However, there should be no ambiguity between the two cases.

The Batched BLAS interface proposal (Dongarra 2018) takes the second option: it simply omits batched reductions. In some cases, users could replace the missing features with other functions. For example, a batched dot product $x^T y_1$, $x^T y_2$, …, $x^T y_K$ could be expressed as a non-batched matrix-vector product, and a batched dot product ${x_1}^T y_1$, ${x_2}^T y_2$, …, ${x_K}^T y_K$ could be expressed as a batched matrix multiply, where each problem in the batch is 1 by N times 1 by N (where N is the number of elements in each vector). However, this approach has four issues.

1. The authors' experience is that special cases of more general BLAS functions (e.g., 1 by N times 1 by N matrix multiplies) may not perform as well as more specialized BLAS functions (e.g, dot products).

2. Some reduction functions cannot naturally be represented this way. The max-norm (or index of the max absolute value, which is what the BLAS computes) is an example.

3. Other reduction functions could be represented with existing operations, but doing so efficiently may be difficult. For example, a batched 1-norm could be represented as the dot product of an elementwise absolute value of a matrix, with a vector whose elements are all the value 1. One could represent the elementwise absolute value lazily with an accessor, and a vector of all ones efficiently with a nonunique layout. However, the resulting code might not be efficient.

4. Even if some batched reduction functions could be represented with existing operations, doing so may sacrifice accuracy or correctness guarantees due to rounding error. For example, the batched 2-norm could be represented as a batched dot product of each vector by itself, followed by an elementwise square root. However, the result would be more prone to underflow or overflow, since the batched dot product would be working with squares of elements.

§ 4.2.5 Representing broadcast parameters

If the output `mdspan` has an extra rank, then we assume that users want to do a batched computation, and we treat the leftmost extent as the batch extent (the index of a problem

in the batch). Any input `mdspan` with an extra rank also has its leftmost extent treated as the batch extent.

Users may want to repeat a single input for all the problems in a batch. For example, users may want to perform matrix-vector multiply with the same matrix, but different input and output vectors. We call the repeated single input a "broadcast" parameter. This feature minimizes storage requirements and overhead. Output (or input/output) `mdspan` cannot be broadcast.

We represent broadcast input parameters as an input `mdspan` without the extra batch extent. That is, the input's rank is the same as it would have been in the non-batched case. This interpretation is unambiguous for all P1673 functions.

We could have chosen to express broadcast parameters by requiring that they have the "extra" batch extent, but using a nonunique layout: for example, a strided layout with stride zero in the batch mode. (`layout_stride` does not permit this, but general strided layouts can have zero strides.) Users can still do this with our proposal. However, we consider nonunique layouts an expert mdspan feature that is more challenging for users to implement. We also do not want to add such layouts to the Standard Library, as we think broadcast parameters have a natural representation without them.

§ 5 Wording Sketch

Here is an example of the wording for non-batched functions in P1673.

```
template<class ExecutionPolicy,
 in-matrix InMat,
 in-vector InVec,
 out-vector OutVec>
void matrix_vector_product(ExecutionPolicy&& exec,
  InMat A,
  InVec x,
  OutVec y);
```

Effects: Computes $y = Ax$

This wording relies on exposition-only concepts *`in-matrix`*, *`in-vector`*, and *`out-vector`*, given by the following.

```
template<class T>
concept in-matrix = // exposition only
  is-mdspan<T>::value &&
  T::rank() == 2;

template<class T>
concept in-vector = // exposition only
is-mdspan<T>::value &&
T::rank() == 1;

template<class T>
```

```
concept out-vector = // exposition only
is-mdspan<T>::value &&
T::rank() == 1 &&
is_assignable_v<typename T::reference, typename T::element_type> &&
T::is_always_unique();
```

We propose supporting the batched case by adding new overloads of the function with new exposition-only concepts. These concepts account for possible broadcasting of input arguments.

```
template<class T>
concept batched-in-matrix = // exposition only
is-mdspan<T>::value &&
(T::rank() == 2 || T::rank() == 3);

concept batched-in-vector = // exposition only
is-mdspan<T>::value &&
(T::rank() == 1 || T::rank() == 2);

template<class T>
concept batched-out-vector = // exposition only
is-mdspan<T>::value &&
T::rank() == 2 &&
is_assignable_v<typename T::reference, typename T::element_type> &&
T::is_always_unique();
```

The concepts *`batched-out-***`* and *`batched-inout-***`* strictly increase the rank requirement by one, while the input concepts *`in-***`* permit either the original non-batched rank (for broadcasting) or one rank higher.

In addition to these new exposition-only concepts, we will need some exposition-only helper functions. We emphasize that high-performance implementations likely will not just be a parallel loop over calls to these functions.

```
template<batched-in-vector InVec>
requires(InVec::rank() == 1)
auto get_batch_vector(InVec v,
  typename InVec::index_type /* batch */) {
  return v;
}

template<batched-in-vector InVec>
requires(InVec::rank() == 2)
auto get_batch_vector(InVec v, typename InVec::index_type batch) {
  return submdspan(v, batch, full_extent);
}

template<batched-out-vector OutVec>
auto get_batch_vector(OutVec v, typename OutVec::index_type batch) {
  return submdspan(v, batch, full_extent);
}
```

With those functions (and their equivalents for matrices) we can now define the batched overload for `matrix_vector_product`.

```
template<class ExecutionPolicy,
 batched-in-matrix InMat,
 batched-in-vector InVec,
 batched-out-vector OutVec>
void matrix_vector_product(
  ExecutionPolicy&& exec,
  InMat A,
  InVec x,
  OutVec y);
```

Preconditions: `(y.extent(0) == x.extent(0) || x.rank() == 1) && (y.extent(0) == A.extent(0) || A.rank() == 2)` is `true`.

Effects: Equivalent to calling `matrix_vector_product(get_batch_matrix(A, i),get_batch_vector(x, i),get_batch_vector(y, i))` for each `i` in the range of [0,`A.extent(0)`).

This leaves the question of overloads with a separate scaling factor per subobject. To support that, we would add an additional overload taking a rank-1 `mdspan` with the scaling factors.

```
template<class ExecutionPolicy,
  in-vector Alphas,
  batched-in-matrix InMat,
  batched-in-vector InVec,
  batched-out-vector OutVec>
void matrix_vector_product(
  ExecutionPolicy&& exec,
  Alphas alphas,
  InMat A,
  InVec x,
  OutVec y);
```

Preconditions: `(y.extent(0) == alphas.extent(0)) && (y.extent(0) == x.extent(0) || x.rank() == 1) && (y.extent(0) == A.extent(0) || A.rank()==2)` is `true`.

Effects: Equivalent to calling `matrix_vector_product(scaled(alphas[i],get_batch_matrix(A, i)),get_batch_vector(x, i),get_batch_vector(y, i))` for each `i` in the range of [0,`A.extent(0)`).

The fact that the output `mdspan` for the batched case always has one higher rank, resolves any possible ambiguity between the non-batched overloads (that never take scaling factor parameters, as scaling factors always come in through the result of `scaled`) and the batched overloads (that either take no scaling factors, or take all the scaling factors explicitly as `mdspan`). For example, here is a non-batched updating `matrix_vector_product` overload.

```
template<class ExecutionPolicy,
         in-matrix InMat,
         in-vector InVec1,
         in-vector InVec2,
         out-vector OutVec>
void matrix_vector_product(ExecutionPolicy&& exec,
                           InMat A,
                           InVec1 x,
                           InVec2 y,
                           OutVec z);
```

It has five parameters, and the last has rank 1. Here is a batched overwriting overload. Note that it only takes `alphas`, not `betas`, because `betas` would only apply to the updating case.

```
template<class ExecutionPolicy,
  in-vector Alphas,
  batched-in-matrix InMat,
  batched-in-vector InVec,
  batched-out-vector OutVec>
void matrix_vector_product(
  ExecutionPolicy&& exec,
  Alphas alphas,
  InMat A,
  InVec x,
  OutVec y);
```

While it also takes five parameters, the last parameter has rank 2, so overload resolution is not ambiguous beteween them. Finally, here is a batched updating overload.

```
template<class ExecutionPolicy,
  in-vector Alphas,
  batched-in-matrix InMat,
  batched-in-vector InVec1,
  in-vector Betas,
  batched-in-vector InVec2,
  batched-out-vector OutVec>
void matrix_vector_product(
  ExecutionPolicy&& exec,
  Alphas alphas,
  InMat A,
  InVec1 x,
  Betas betas,
  InVec2 y,
  OutVec z);
```

It takes seven parameters, and the last parameter always has rank 2. Functions either take no explicit scaling factors, or all of the scaling factors that they need to take, so we do not need to consider cases where either `alphas` or `betas` are omitted, but not both.

§ 6 References

— Samuel D. Relton, Pedro Valero-Lara, and Mawussi Zounon, “A Comparison of Potential Interfaces for Batched BLAS Computations,” NLAFET Working Note 5, August 2016.

— Jack Dongarra, Iain Duff, Mark Gates, Azzam Haidar, Sven Hammarling, Nicholas J. Higham, Jonathan Hogg, Pedro Valero Lara, Piotr Luszczek, Mawussi Zounon, Samuel D. Relton, Stanimire Tomov, Timothy Costa, and Sarah Knepper, “Batched BLAS (Basic Linear Algebra Subprograms) 2018 Specification,” July 2018.

§ 7 Acknowledgements

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC., a wholly owned subsidiary of Honeywell International, Inc., for the U.S. Department of Energy’s National Nuclear Security Administration under contract DE-NA-0003525.

This work was supported by the Exascale Computing Project (17-SC-20-SC), a collaborative effort of the U.S. Department of Energy Office of Science and the National Nuclear Security Administration.